Edge Intelligence
06-03-2026

P.Raghu
25MML0029

Number Plate Detection System

1. Introduction

The Number Plate Detection System is designed to identify and extract vehicle registration numbers from images in real time using machine learning and Optical Character Recognition (OCR). The backend is implemented using Spring Boot with Java, following the MVC (Model-View-Controller) architecture. It communicates with the frontend through RESTful APIs, which serve as the entry point for all client requests. When a user uploads a vehicle image, the backend validates it, stores it on an FTP server, processes it through a detection model, and returns the extracted number plate text.

2. System Architecture

The backend consists of the following core components working together as a pipeline:

- **REST API Layer:** Entry point for all requests; exposes endpoints for image upload and result retrieval.
- **Controller Layer:** Validates incoming requests (file format, size, parameters) and delegates to the service layer.
- **Service Layer:** Core business logic - image validation, metadata generation, FTP upload, and detection model invocation.
- **FTP Server:** Centralized image storage that reduces application server load and enables secure file management.
- **Detection Model (AI/ML):** Identifies number plate regions using bounding boxes and extracts text via OCR.
- **PostgreSQL Database:** Stores detection results and system logs using Hibernate ORM.

```
+---------------------------+
|   Frontend / Camera       |
+-------------+-------------+
            |  HTTP POST /upload
                      v
+---------------------------+
|  REST API (Spring Boot)   |
|    Controller Layer       |
+-------------+-------------+
                      |
                      v
+---------------------------+
|     Service Layer         |
|    (Business Logic)       |
 +------+-----------+-------+
            |           |
            v           v
+-------------+ +----------------+
| FTP Server  | | Detection Model|
| (Image      | | (AI/ML + OCR)  |
|  Storage)   | |                |
+------+------+ +-------+--------+
           |                |
           +--------+-------+
                       v
+---------------------------+
|  PostgreSQL Database      |
|  (Results & Logs)         |
+---------------------------+
```

3. Backend Workflow

The end-to-end processing pipeline operates as follows:

Client Request: The frontend captures or selects a vehicle image and sends it to the backend via HTTP POST /upload as multipart form data.

Controller Validation: The Spring Boot controller validates the file format, size, and request integrity. Invalid requests are rejected immediately with an appropriate error response.

Service Layer Processing: The service layer generates metadata (filename, timestamp), transfers the image to the FTP server, invokes the detection model, and logs results to the database. It also verifies FTP and database connectivity before processing.

FTP Storage: The image is uploaded to a centralized FTP server, storing the image name, upload timestamp, and file path. This offloads storage from the application server.

Number Plate Detection: The ML model loads the image, detects the plate region with a bounding box, and applies OCR to extract alphanumeric characters (e.g., AP07AB1234).

Database Logging: Detection results, processing status, and logs are stored in PostgreSQL for tracking, debugging, and auditing.

Response: A JSON response containing the filename, detected plate number, status, and timestamp is returned to the client.

4. Database Schema

Field	Data Type	Description
filename	VARCHAR	Uploaded image name
upload_time	TIMESTAMP	Date and time of upload
detected_plate	VARCHAR	Extracted number plate text
processing_status	VARCHAR	Success or failure
server_logs	TEXT	System logs for the request

5. Sample API Response

```
{
    "filename"    : "vehicle1.jpg",
    "plate_number": "AP07AB1234",
    "status"      : "success",
    "upload_time" : "2026-03-06 06:20:00"
}
```

6. Parallel Processing & Deployment

Spring Boot provides built-in multithreading support, allowing multiple users to upload and process images simultaneously without queuing delays. Each request is handled by a separate thread, ensuring consistent response times under concurrent load.

The application is packaged as a standalone JAR file and can be deployed with a single command: java -jar number-plate-detection.jar. On startup, the Spring Boot server initializes, database and FTP connections are established, and all API endpoints become active. The system is then ready to receive and process requests without any manual intervention.

7. Conclusion

The backend architecture provides a scalable and efficient framework for real-time number plate detection. The integration of Spring Boot, FTP server, PostgreSQL, and a machine learning-based detection model ensures reliable operation across the full pipeline - from image upload to result delivery. The modular design supports future extensions such as real-time video processing, multi-language plate recognition, and integration with external vehicle databases.